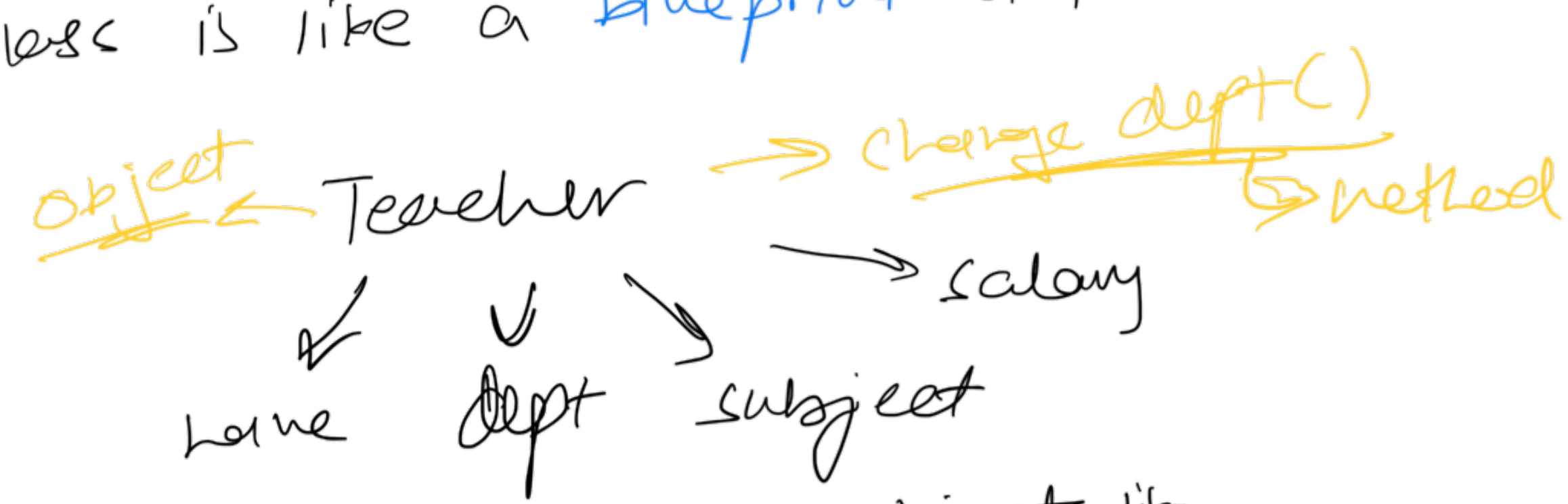
Object Oriented Programming

#Classes & Objects

- objects are entities in the real world
- class is like a blueprint of these entities



we need not make many objects like Teacher1, Teacher2, ...
we can reuse the code

#Access Modifiers

- **private** - data & methods accessible inside class
- **public** - data & methods accessible to everyone
- **protected** - data & methods accessible inside class & to its derived class

#Encapsulation

- wrapping up of data & member fns in a single unit called class
- helps in data hiding (use private for sensitive info)

#Constructor

- special method invoked automatically at time of object creation
- used for initialization
- same name as class
- constructor doesn't have a return type
- only called once (automatically) at object creation
- memory allocation happens when constructor is called

#this pointer

- special pointer that points to the current object

  obj.fun()

  if this function has to refer to obj ki properties it can access by writing prop
  we can also write
  this -> prop
- "hey function, i'm the one calling you, here is my address if you need it"

#copy constructor

- used to copy properties of one object into another

★ Shallow & Deep copy

Shallow - this copy of an object copies all of the member values from one object to another

Deep - this copy not only copies the member values but also makes copies of any dynamically allocated memory that the members point to

#Destructor

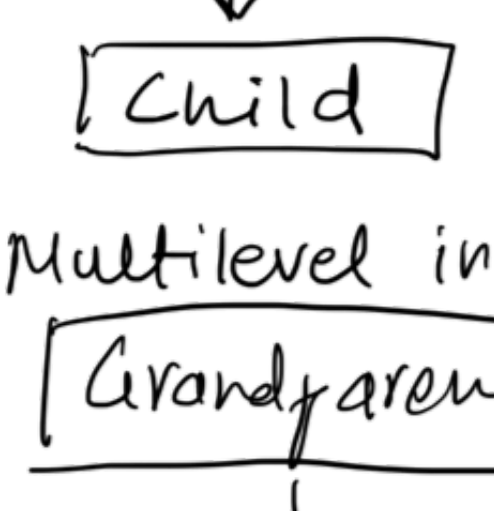
- deallocates memory

#Inheritance

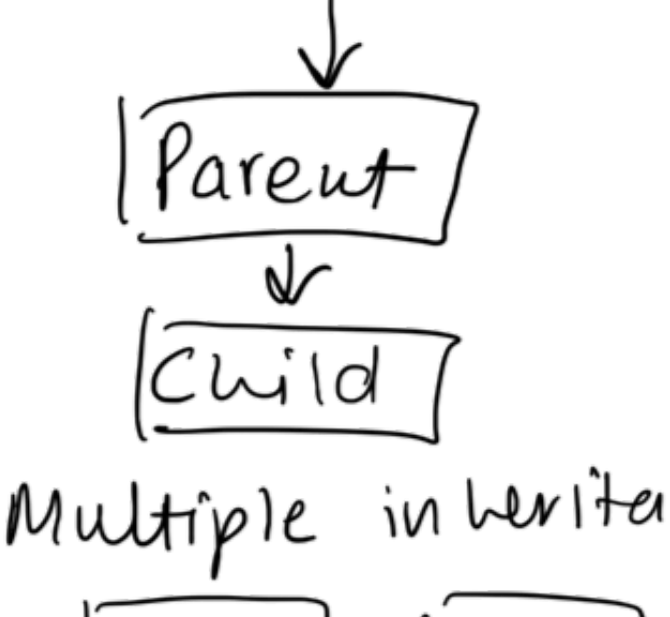
- one class can use properties & behaviours of another class
- class that gives properties : Base Class
- class that receives them : Derived class

Base Class	Derived class		
	Private mode	Protected mode	Public mode
Private	Not inherited	Not inherited	Not inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

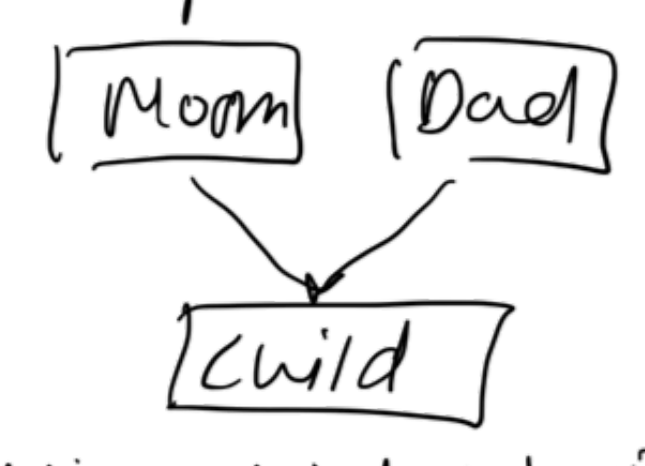
- types
  1. Single inheritance



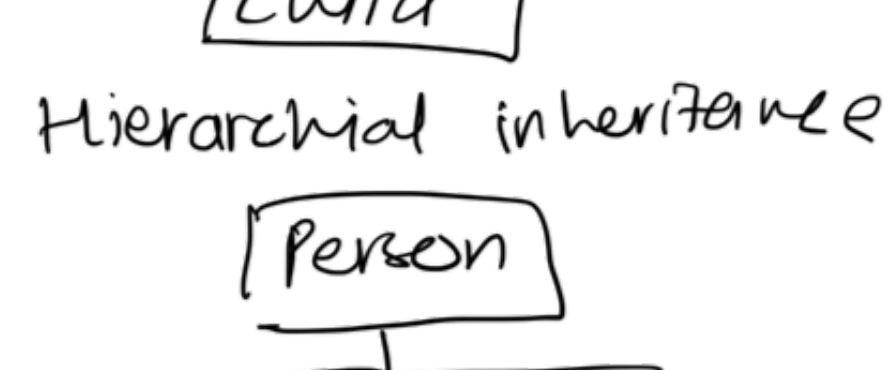
  2. Multilevel inheritance



  3. Multiple inheritance



  4. Hierarchial inheritance



  5. Hybrid inheritance
     - mix of above

#Polymorphism

- ability of objects to take on different forms or behave in different ways depending on the context in which they are used
- types
  1. Compile Time
     - Constructor overloading - either parameterized or non parameterized depending on context
     - Function overloading

```
class {
    fun( )   <- params
    fun( )   <- diff params
}
```

       same name diff params
     - operator overloading
  2. Run time
     - Function overriding
       - parent & child both contain the same function with different implementation
       - parent class function is said to be overridden (child has more priority)
     - Virtual function
       - it's a member function in a base class that you expect to be overridden in a derived class

Abstraction

- hiding all unnecessary details & showing only the important parts
- can use access modifiers or abstract classes

★ Abstract classes
- used to provide a base class from which other classes can be derived
- cannot be instantiated and are meant to be inherited
- typically used to define an interface for derived classes
- BLUEPRINT!

#Static keyword

- Variables declared as static in a function are created & initialised once for the lifetime of the program
- persists for the program's lifetime
- destructor called only after main fn over